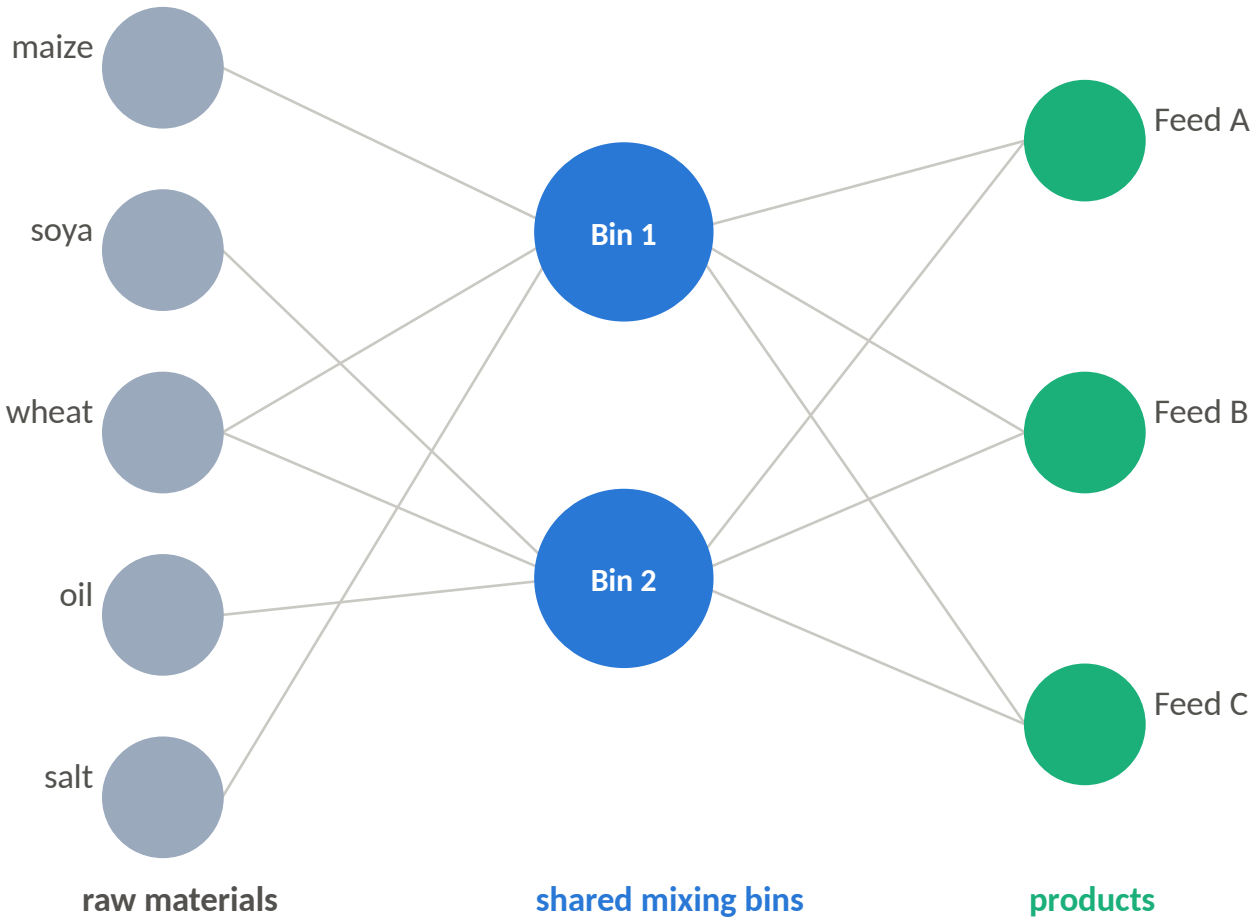
Smarter Restarts for Feed-Blending Optimization

Improving Sequential Linear Programming (SLP) for the pooling problem

IA-2 · Introduction to Optimization Techniques · Dept. of IT, K. J. Somaiya College of Engineering

Group OPT-09: Razzan Tambe (15) · Saurabh Tripathi (04) · Vivek Sharma (12)

The problem: make animal feed at the lowest cost



- A feed mill must deliver fixed orders of many feeds.
- Each feed needs its nutrients (protein, fat, fibre, …) within limits.
- Raw materials are first mixed in a few shared bins.
- **Goal: choose what goes into each bin and each feed so that the total cost is lowest.**

Why it is hard: shared bins create many 'valleys'

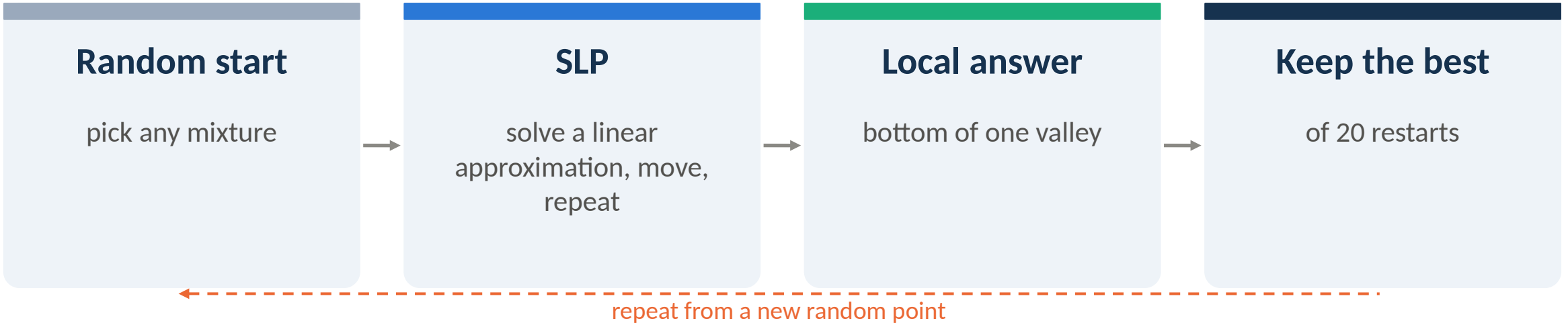
One bin, two feeds

- A bin recipe that is good for Feed A can be bad for Feed B.
- Feed quality = (share of the bin) × (what is in the bin): two unknowns multiplied.
- This makes the problem nonconvex: it has many local best answers.



Many valleys = many local optima. A local method slides into the valley next to where it starts, and the deepest valley is easy to miss.

IA-1: the paper's method (SLP with random restarts)



SLP (Sequential Linear Programming): near the current point the curved problem is replaced by a simple linear one (an LP). Solve the LP, take the step if it really improves, and repeat until it stops improving. It is fast, but it only finds the bottom of the valley it started in.

The gap we found in IA-1

6%

of runs reach a Good solution on the 6 hard instances

12%

of all runs end infeasible (break a constraint)

0

information reused between restarts: every start is random

Good solution = cost within 0.2% of the best cost ever found for that instance.

Gap: random restarts waste most of the time in bad valleys. Yet the problem has a simple structure that nobody used: if the bins are fixed, the rest is an easy LP.

IA-2: restart smart, not random

1

Smart start

Compute each feed's ideal recipe (a small LP), then group similar recipes into the bins.

2

Fix one, solve the other

With the bins fixed, the rest is an easy LP. We use it to repair and polish every answer.

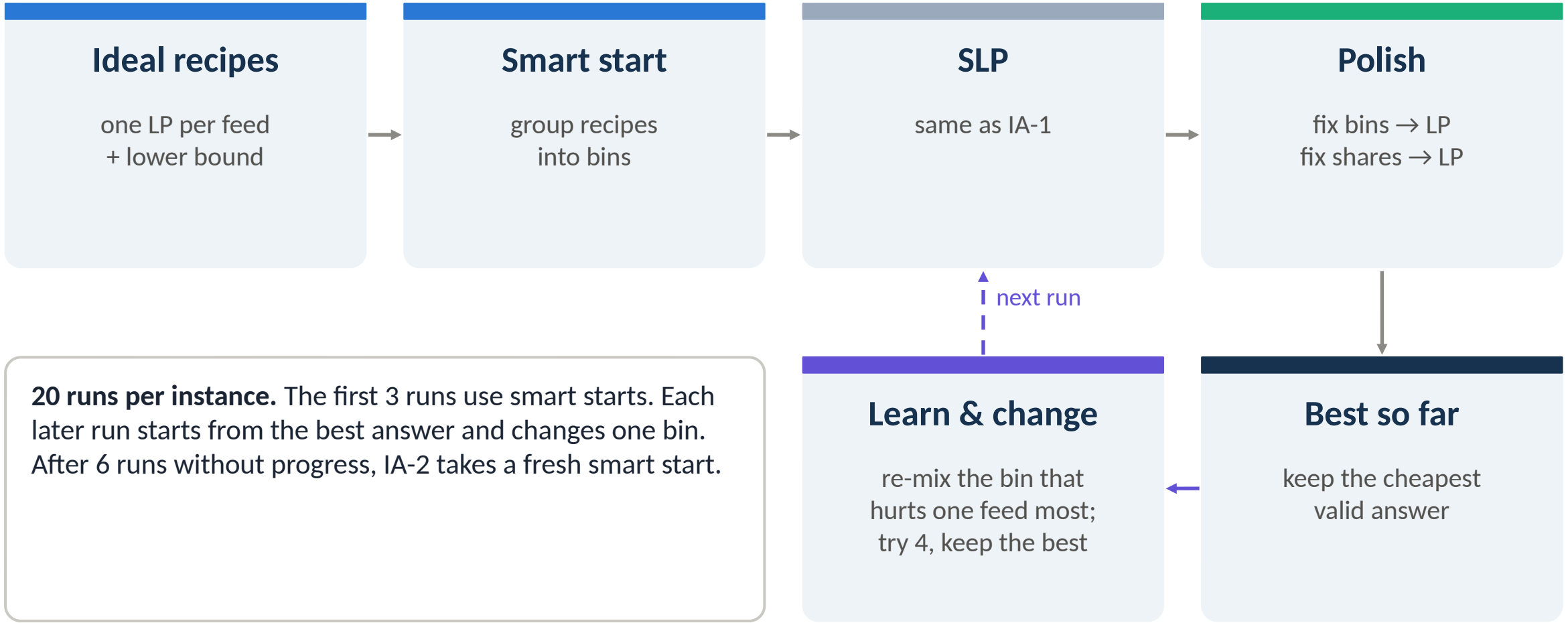
3

Learn from the best

The next run starts from the best answer so far and changes the bin that costs one feed the most.

The model and the SLP solver from IA-1 stay exactly the same: only the starting points and the restarts change.

How IA-2 runs



20 runs per instance. The first 3 runs use smart starts. Each later run starts from the best answer and changes one bin. After 6 runs without progress, IA-2 takes a fresh smart start.

The key trick: fix one part, and the rest is easy

Fix what is in the bins

Then choosing how much of each bin goes into each feed is a plain LP: exact and fast.

Fix how much of each bin goes into each feed

Then choosing what goes into each bin is also a plain LP.

Use: alternating the two LPs repairs broken answers and polishes good ones, and the cost never goes up.

Bonus: ignoring bin sharing gives each feed's ideal recipe and a lower bound on the best cost. On af-1 it equals the bound of the global solver BARON (295.93), in milliseconds.

How we tested IA-1 against IA-2

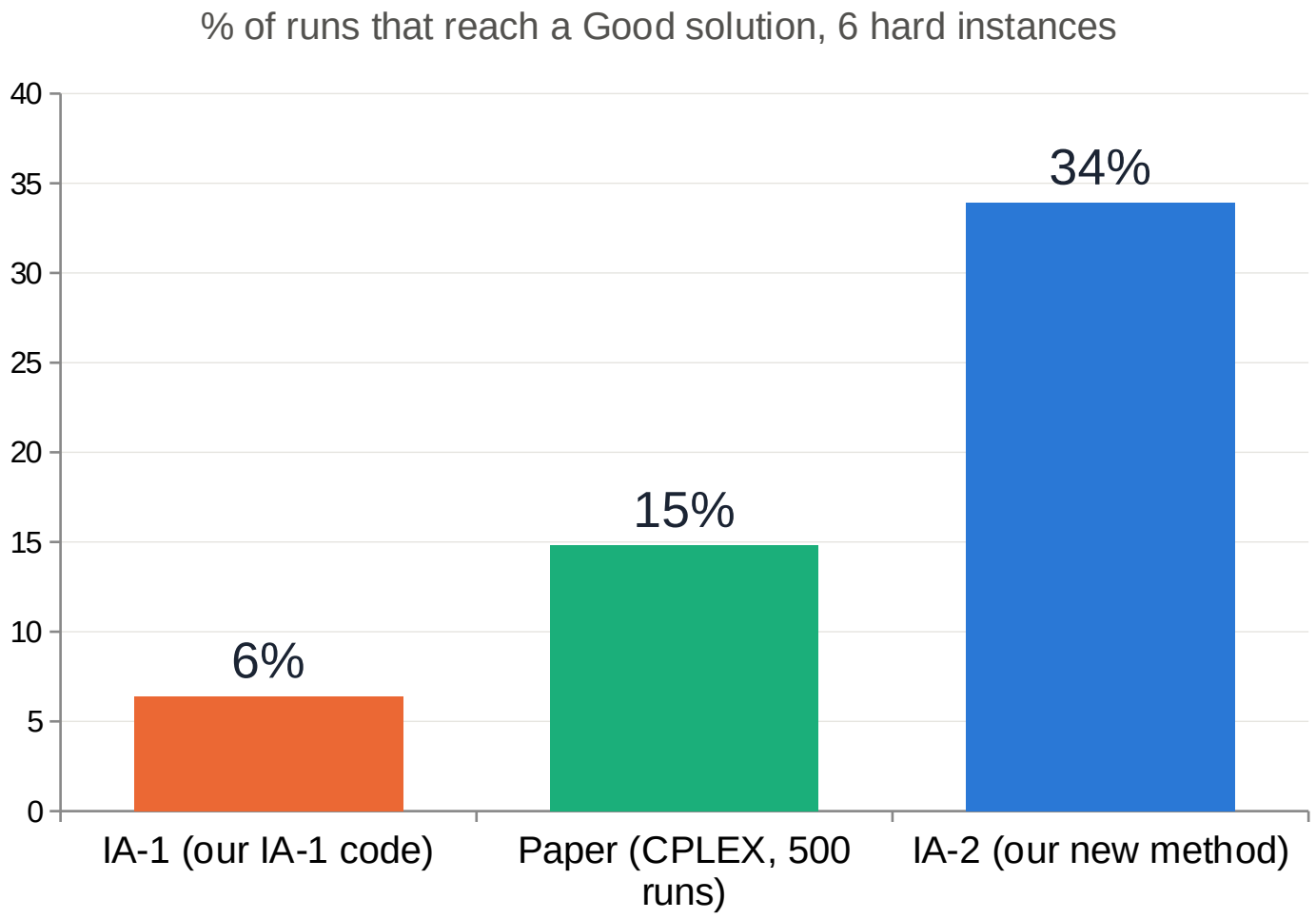
Instances	all 11 real feed-mill problems of the paper (af-1 … af-7b-fs)
Methods	IA-1 = our IA-1 code (baseline) · IA-2 = the new method
Runs	3 seeds × 20 runs per method and instance, same computer
Feasible %	share of runs whose answer meets every constraint
Good %	share of runs within 0.2% of the best cost ever found
Time to Good	expected seconds until a Good answer appears

First check: af-1, the instance we validated in IA-1

af-1	Paper (CPLEX)	IA-1 (our IA-1 code)	IA-2 (new method)
Runs that are Good	79%	100%	100%
Runs that are feasible	100%	100%	100%
Best cost found	296.446	296.446	296.446
Lower bound (proof of quality)	BARON: 295.932 (hours)	none	295.932 (milliseconds)

IA-2 keeps everything IA-1 got right on af-1 and adds a proof: the best cost is at most 0.17% above the true optimum.

Result: IA-2 finds Good solutions 5× more often on hard problems



5×

faster to a Good solution (hard instances)

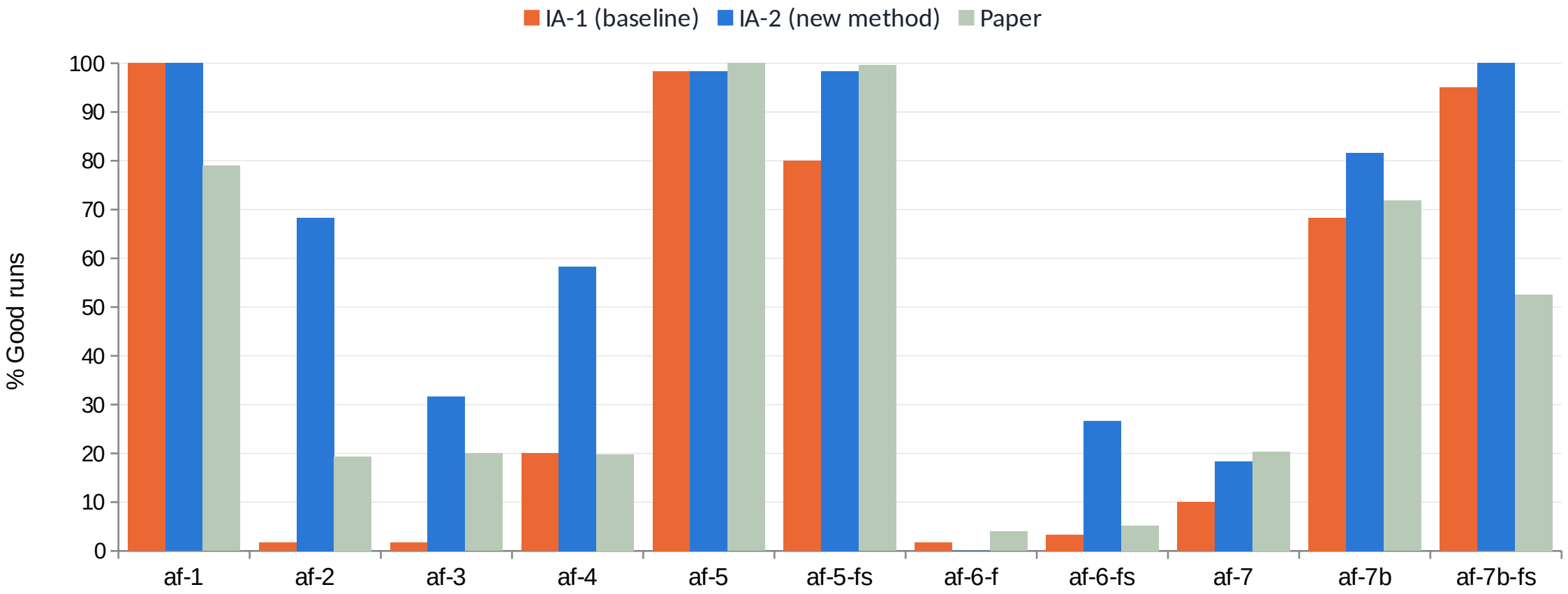
af-2: 2% → 68%

af-3: 2% → 32%

af-4: 20% → 58%

af-6-fs: 3% → 27%

Result on every instance



Easy instances: both near 100%. Hard instances: IA-2 far ahead, except af-6-f, which neither method solves.

Result: more reliable on all 11 instances

88% → 96%

runs that end feasible (IA-1 → IA-2)

44% → 62%

runs that are Good (IA-1 → IA-2)

5 of 11

instances where the gain is statistically significant

Why: smart starts begin in better valleys, the LP polish turns 'almost valid' answers into valid ones, and learning from the best answer means later runs search near it instead of starting over.

Honest limits

af-6-f is not solved	Neither method gets within 0.2% (the paper also only 4%).
Easy problems get a bit slower	The extra LPs cost time when random starts already work (up to ~2×).
af-7 just below the paper	IA-2 18% vs paper 20% (IA-1 10%).

Restarting smartly makes SLP find good feed recipes far more often

Same model and solver as IA-1, plus simple LPs: Good runs on hard problems 6% → 34%, feasible runs 88% → 96%, 5× faster to a Good answer.

Next: choose between smart starts and learning automatically; run the probes in parallel.

Code and demo: GitHub repository + Google Colab notebook. Thank you!

References

[1] A. Grothey, K. McKinnon, “On the effectiveness of sequential linear programming for the pooling problem,” Annals of Operations Research 322 (2023) 691–711.

[2] A. Grothey, K. McKinnon, Pooling problem test data (2020).

[3] C. A. Haverly, “Studies of the behavior of recursion for the pooling problem,” ACM SIGMAP Bulletin 25 (1978).

[4] C. Audet et al., “Pooling problem: alternate formulations and solution methods,” Management Science 50 (2004) 761–776.

[5] H. R. Lourenço et al., “Iterated local search,” Handbook of Metaheuristics (2019).

[6] Group OPT-09, IA-1 report: SLP for nonlinear pooling optimization problems (2026).